Announcements

- Programming Project 4 is assigned
- Midterm 2 Review Session is November 16th, 5pm-7pm, in Haury 129
- The practice midterm has been released on D2L
- Midterm 2 will cover Topics 4 through 7, *including* Topic 5

Topic 8: Beyond Binary Search Trees

By: Professor Lynam

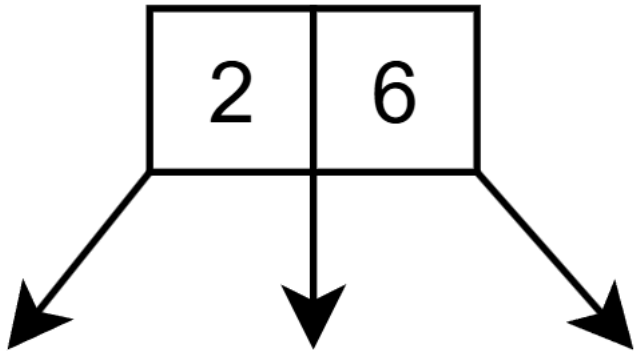
2-3 Trees

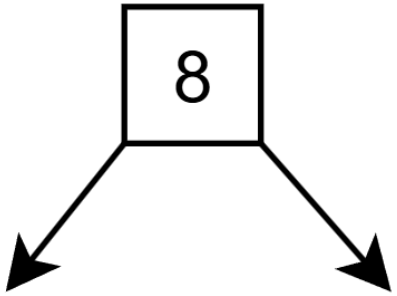
- Is there any reason to store more than one data value in a tree node?
- Yes: we could have a tree with the same data as a binary tree, but it's shorter, which means…
- Pros: Fewer levels to traverse!
- Con: More key comparisons per node

2-3 Trees

- Definition: *2-3 Tree*
- *A 2-3 tree is a balanced, ordered search tree of one or two key values (and two or three child references) per node, such that the tree's growth occurs at its root.*

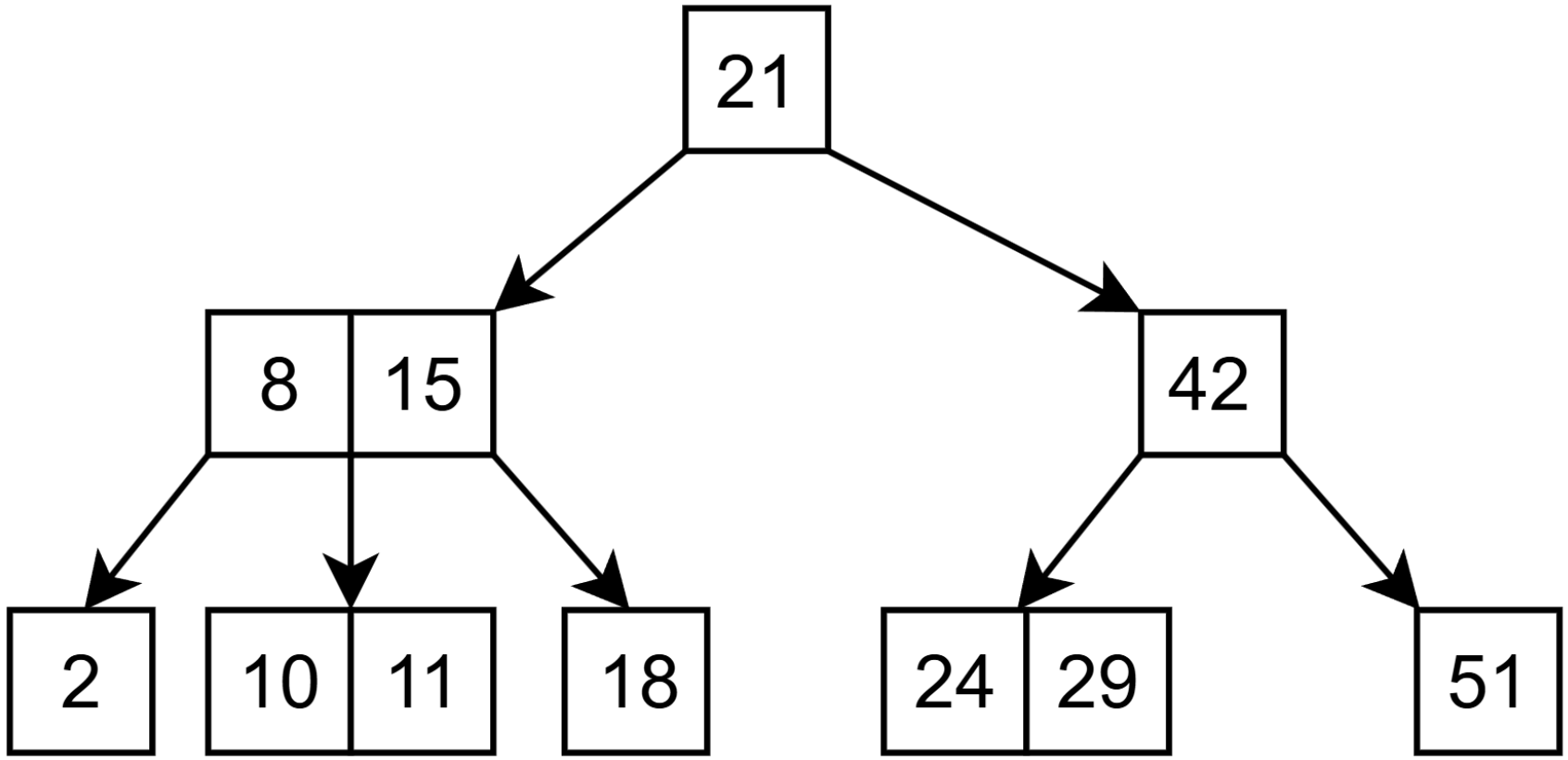
- A 2-node might look like this:
- A 3-node might look like this:





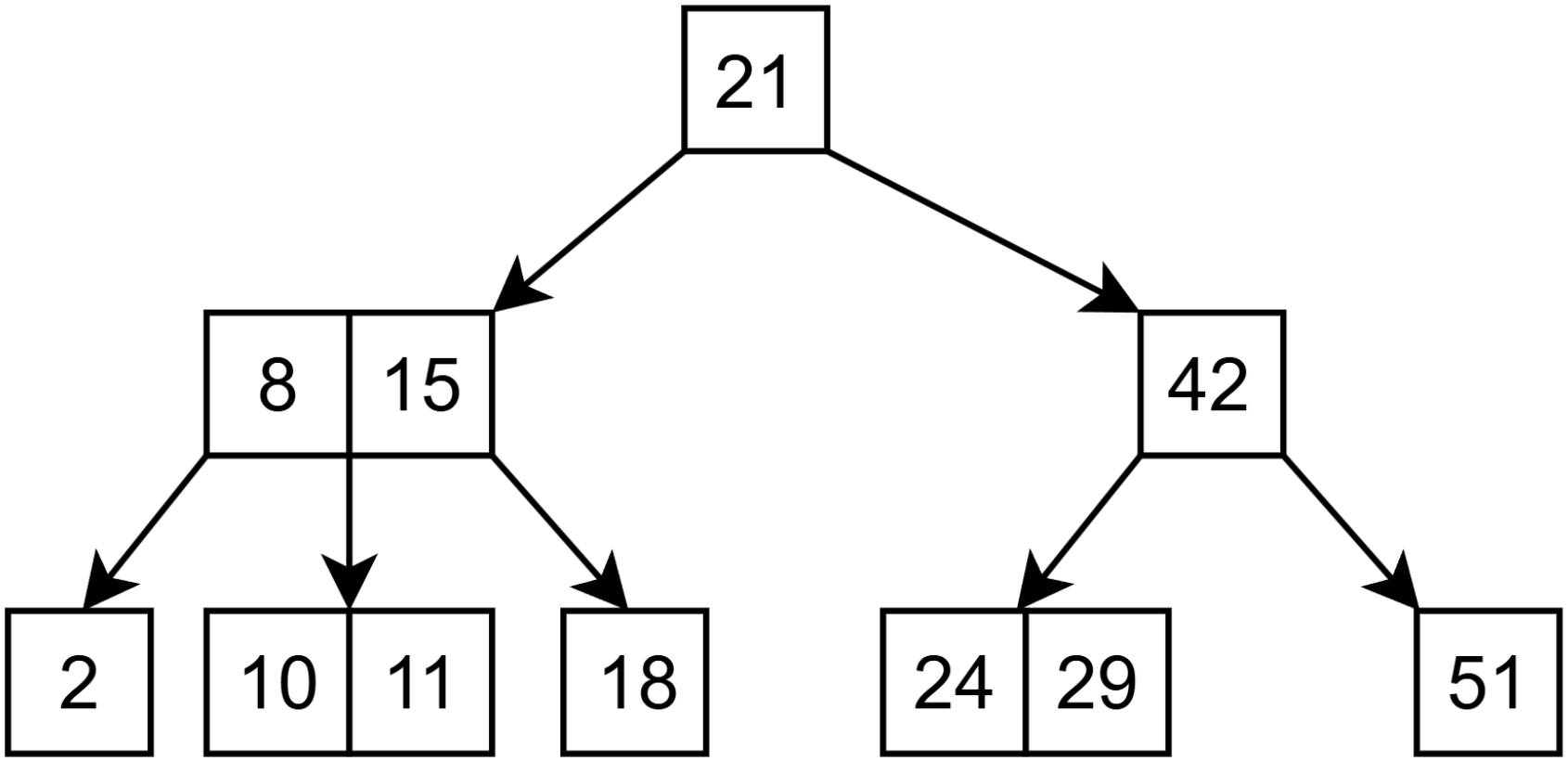
2-3 Trees

- Notes: It's balanced, with all leaves at the same level
- The 2-nodes and 3-nodes can be anywhere in the tree



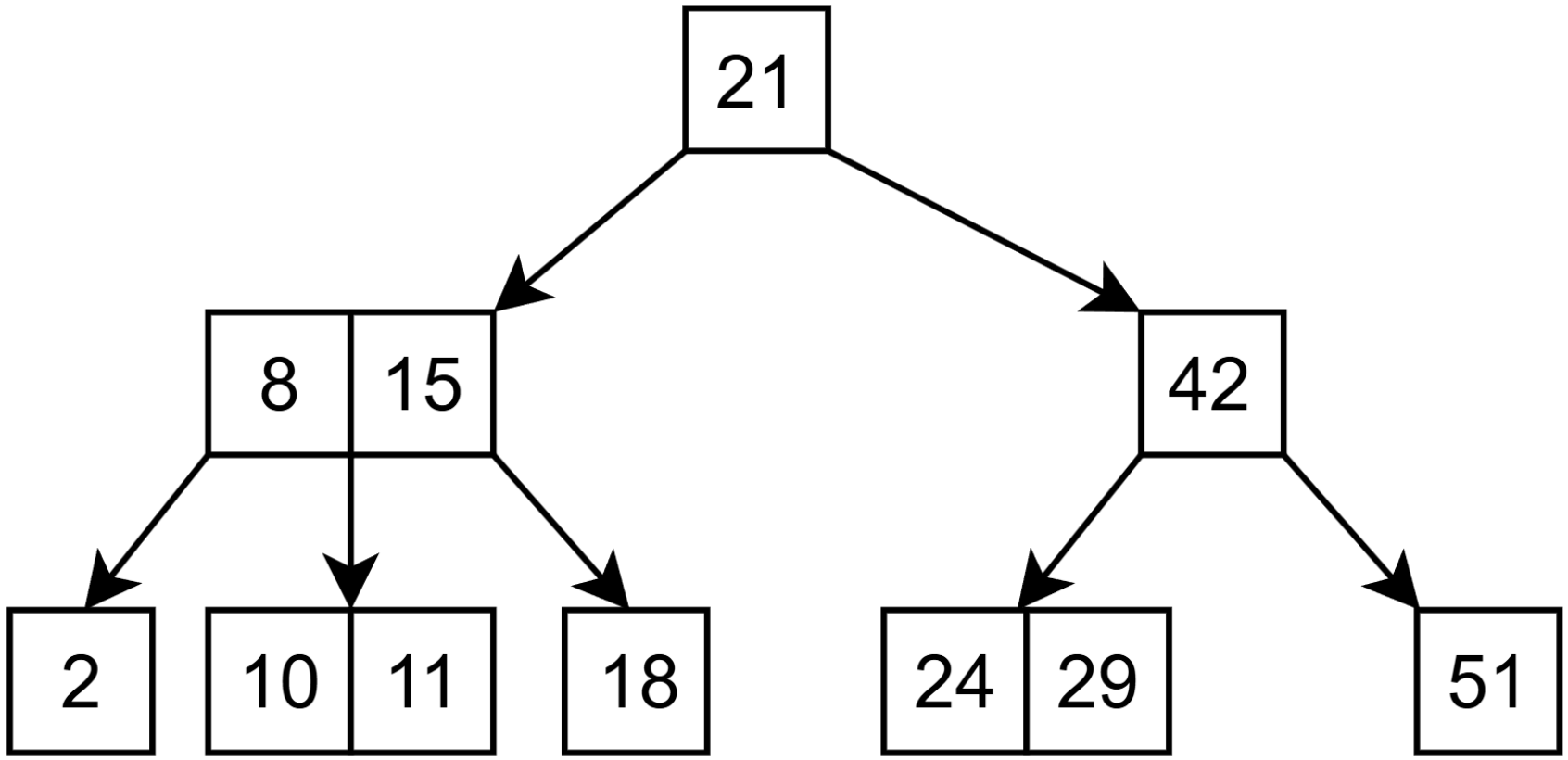
2-3 Trees

- Searching: Find 11.



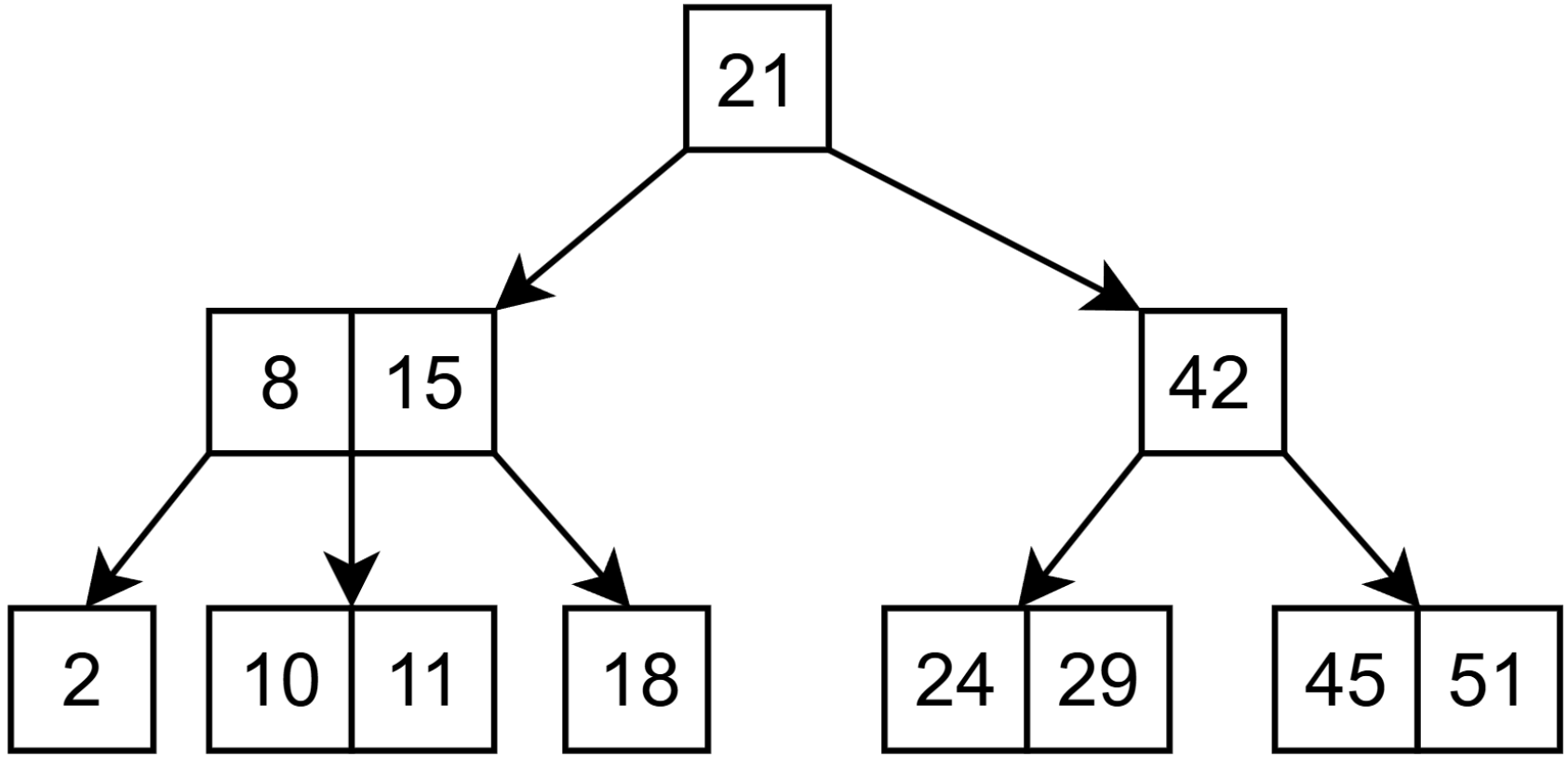
2-3 Trees

- Inserting: Insert 45.
- Start by finding the leaf where you'd expect to find the key.
- If the leaf is a 2-node, add the key (turning it into a 3-node)



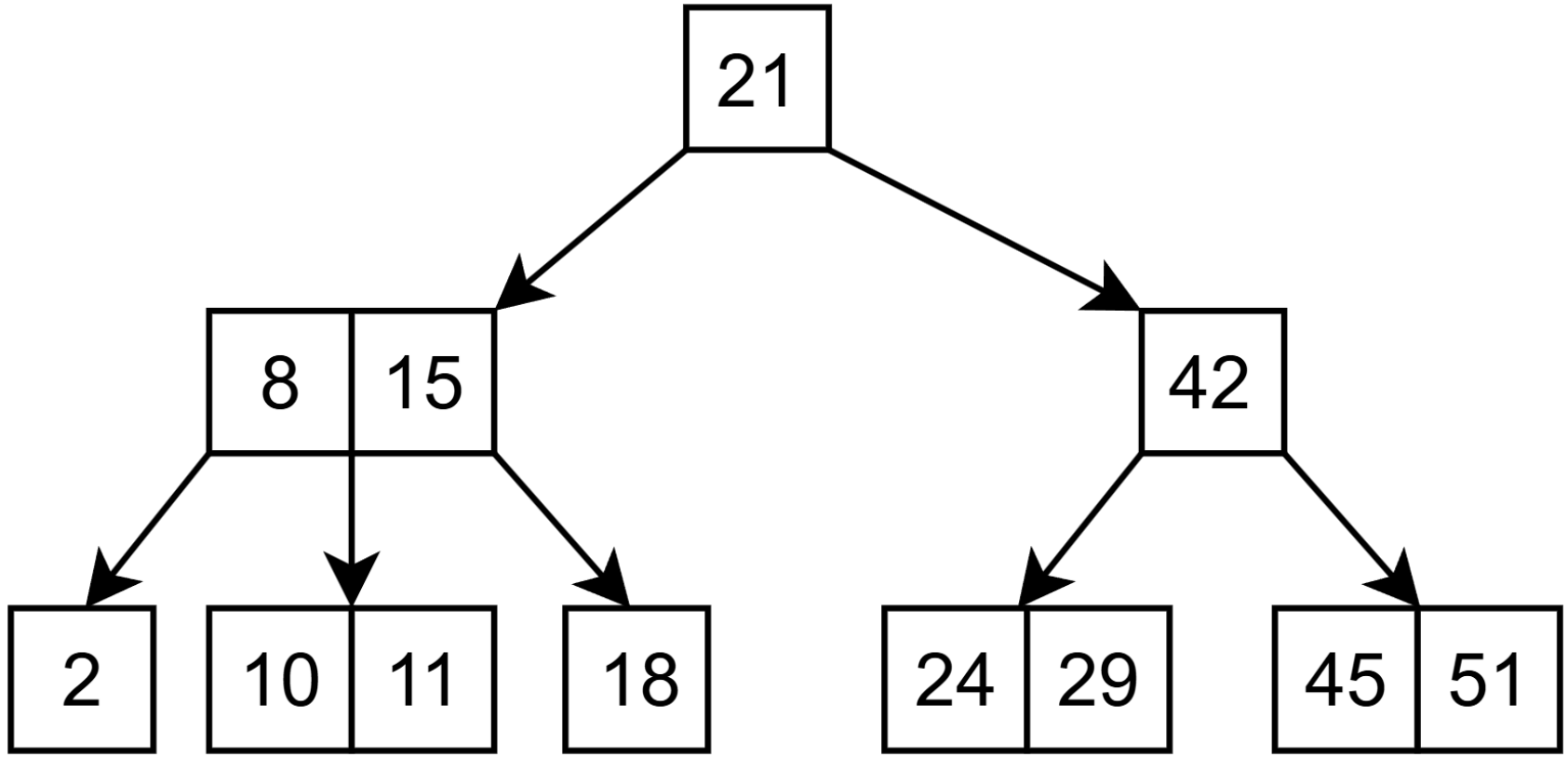
2-3 Trees

- Inserting: Insert 45.
- Start by finding the leaf where you'd expect to find the key.
- If the leaf is a 2-node, add the key (turning it into a 3-node)



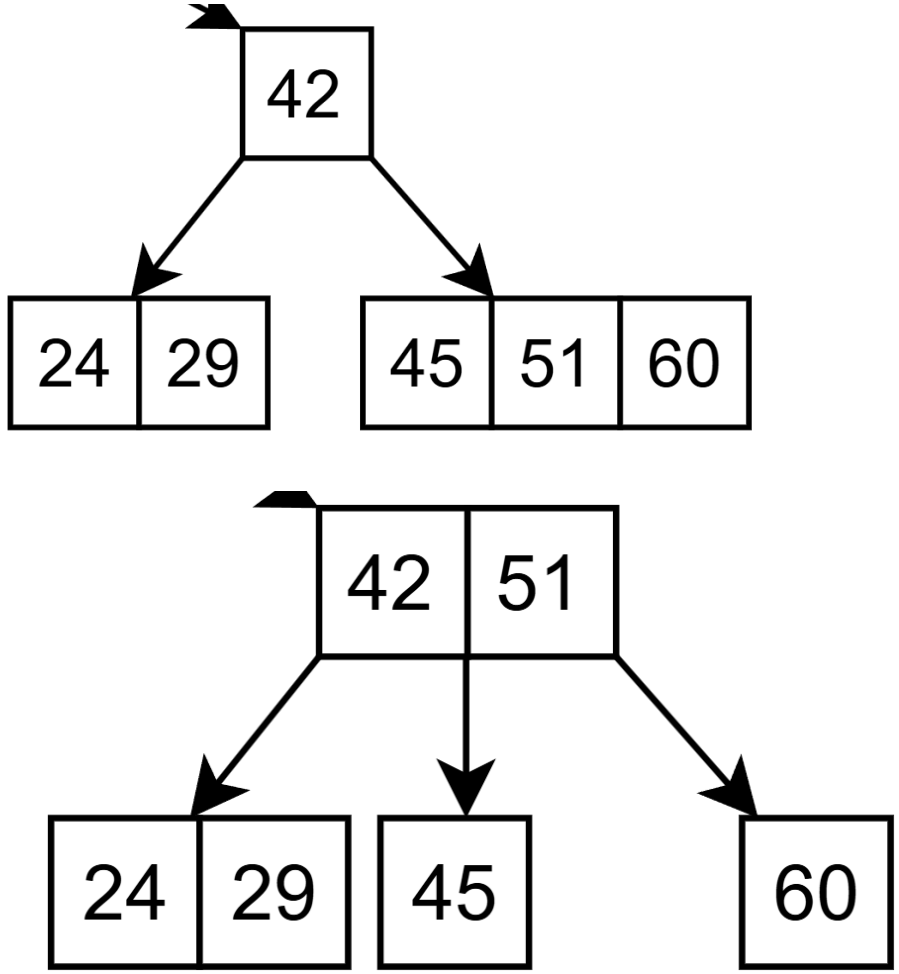
2-3 Trees

- Inserting: Insert 60.
- What if the leaf is already a 3-node?
- Find the median of the 3 values, ‘promote’ it to the parent, make 2-nodes for the other values, and attach them to the parent



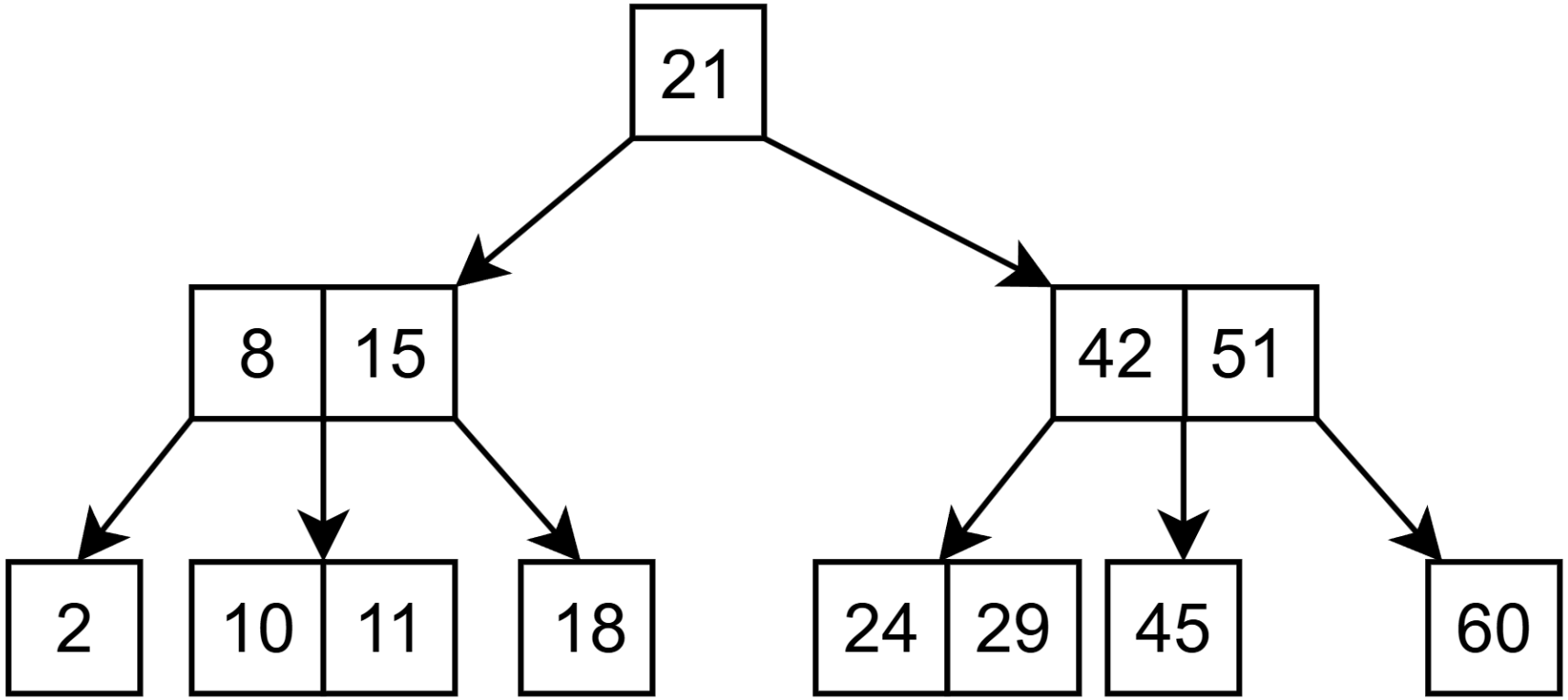
2-3 Trees

- Inserting: Insert 60.
- Find the median of the 3 values, ‘promote’ it to the parent, make 2-nodes for the other values, and attach them to the parent



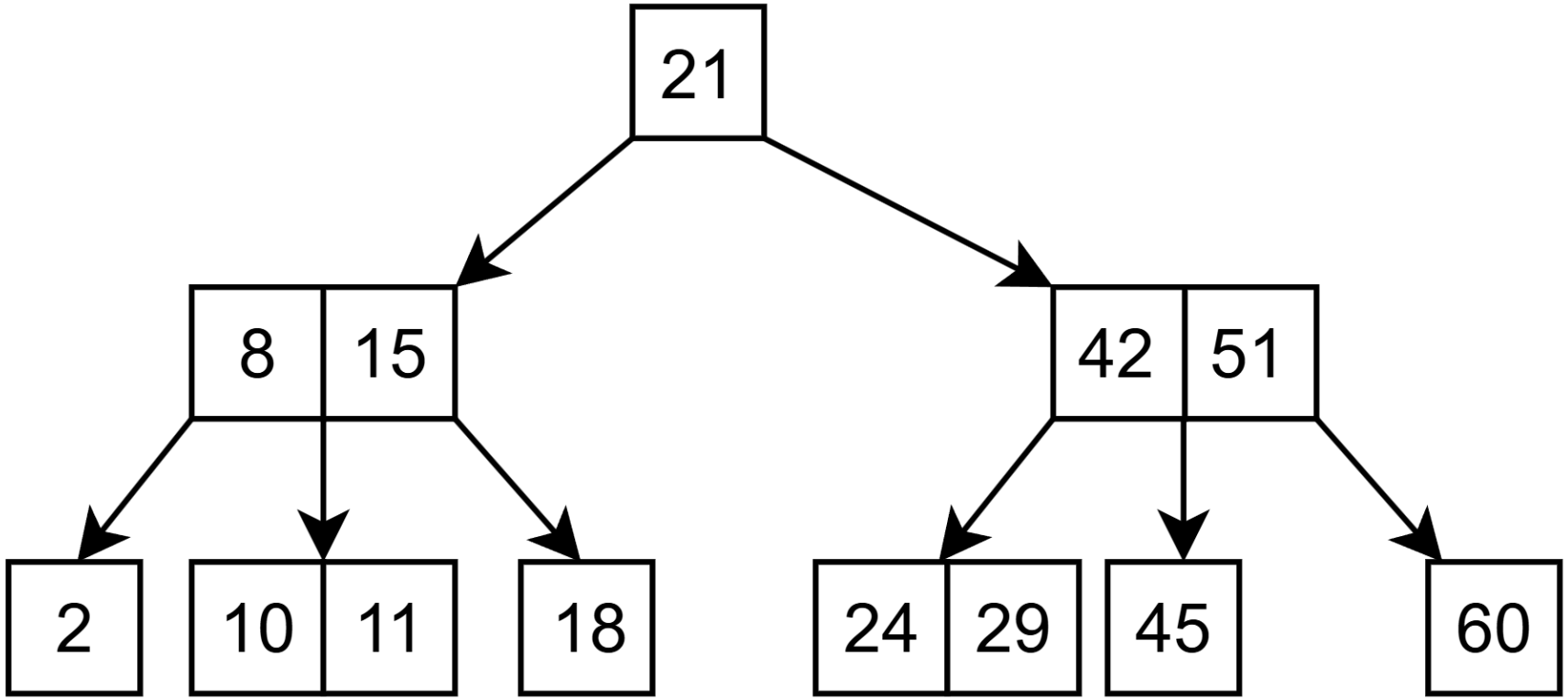
2-3 Trees

- Inserting: Insert 25.
- But what if the parent is already a 3-node?



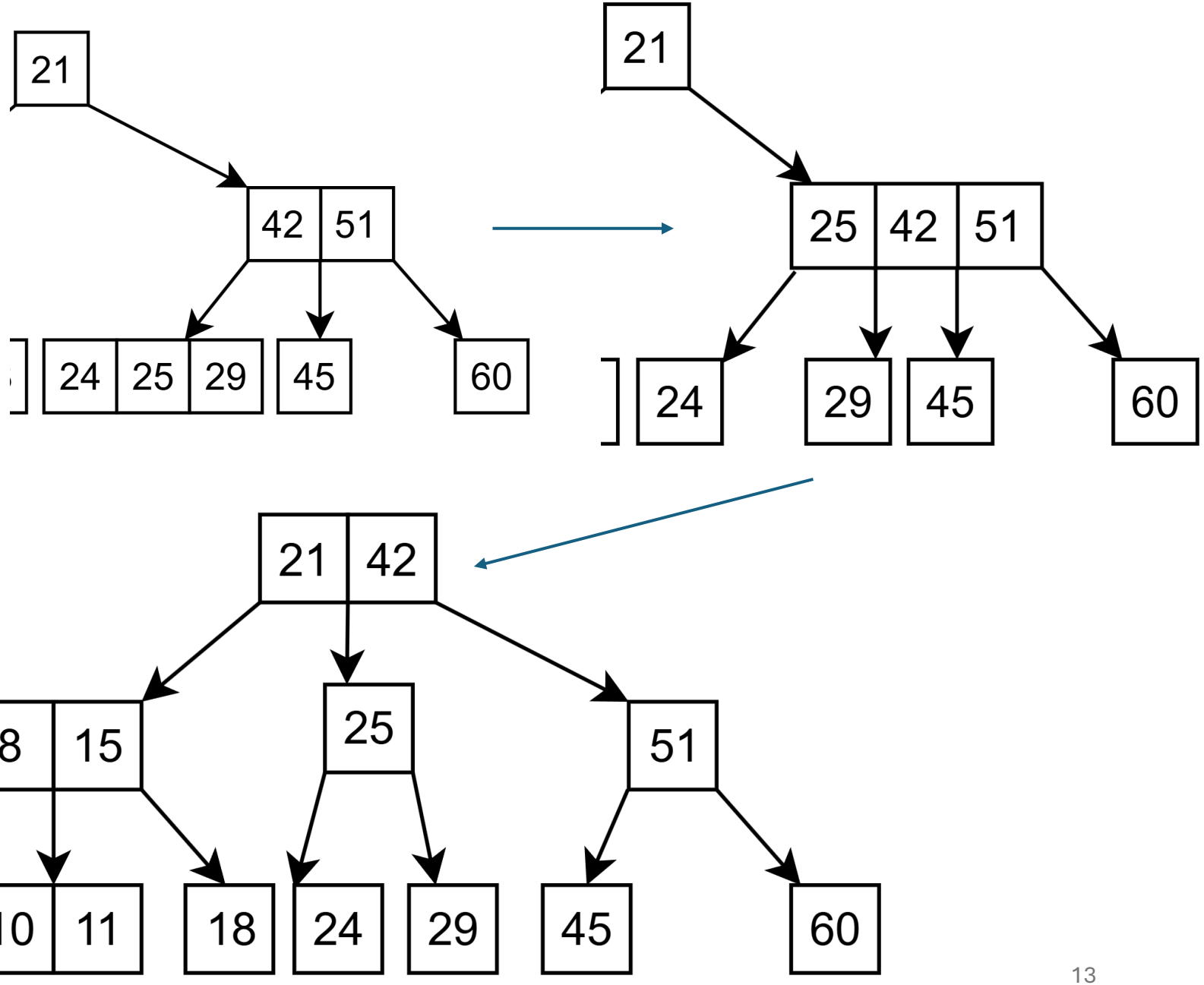
2-3 Trees

- Inserting: Insert 25.
- But what if the parent is already a 3-node?
- Keep promoting based on the medians!



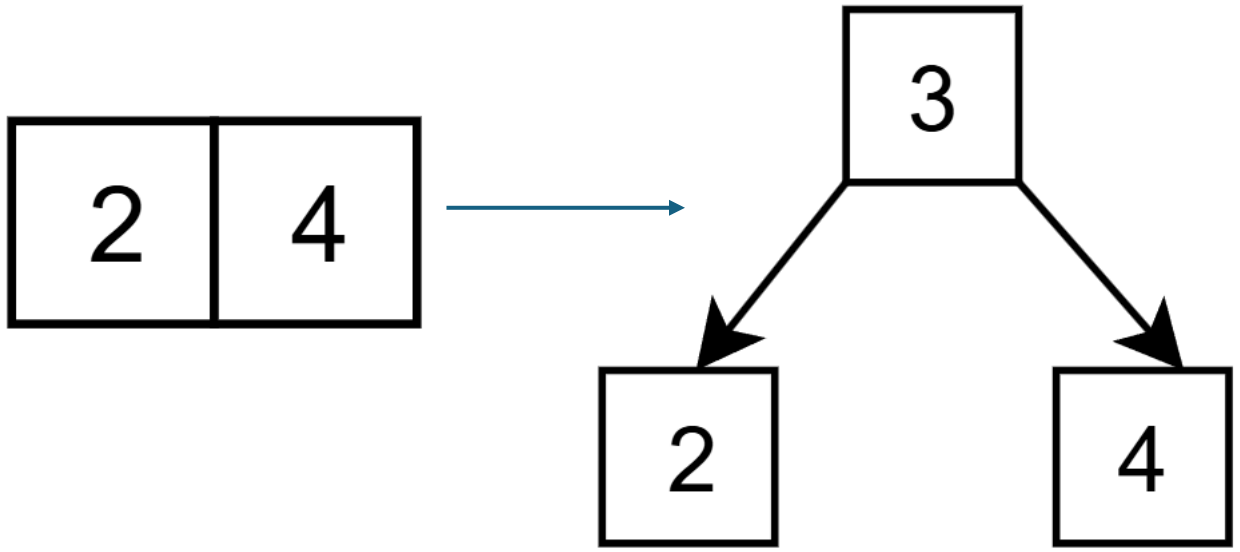
2-3 Trees

- Inserting: Insert 25.
- But what if the parent is already a 3-node?
- Keep promoting based on the medians!



2-3 Trees

- Inserting: Insert 3.
- But what if the root is a 3-node?
- Promote the median into a new root 2-node!



2-3 Trees

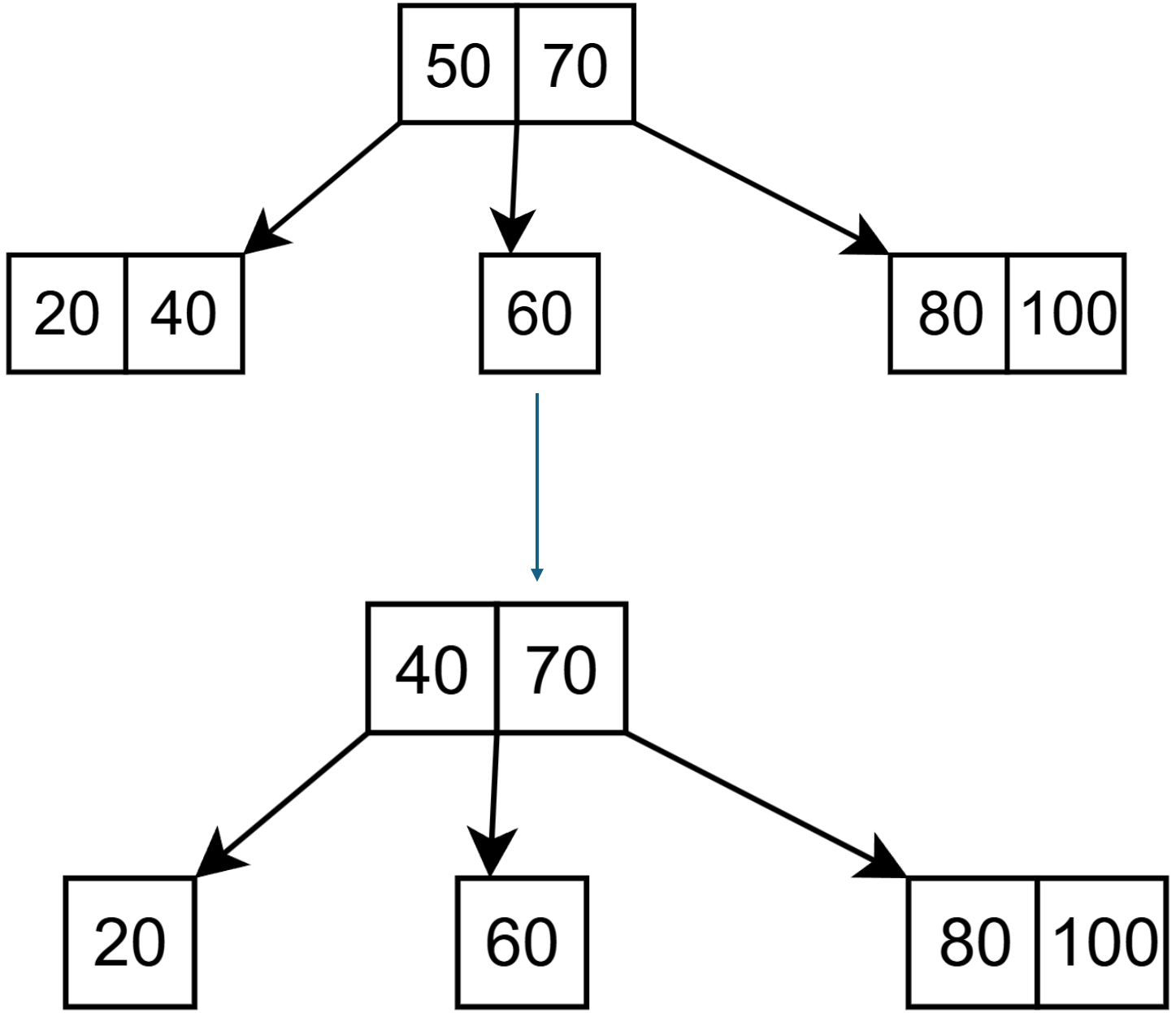
- This ‘promotion’ method allows a 2-3 to grow while maintaining its height balance
- If the root splits, the tree’s height is increased by one, but all the leaves are still on the same level
- If the deletion process (which we’re about to go over) reaches the root and leaves it empty, the tree’s height decreases by one (and the leaves are still all on the same level)

2-3 Trees

2-3 Deletion:
If the victim is an internal node:
    Replace it with its inorder predecessor or successor (repeatedly, till we reach a leaf)
    \\ Note that the deletion is now from leaf node!
If the leaf is a 3-node:
    Leave it in place, but as a 2-node
Else if the leaf has an adjacent sibling 3-node:
    Borrow a key from it (via the parent's separating value)
Else: \\ leaf was 2-node, its adjacent siblings are also 2-nodes
    Collapse the node, one sibling, and the parent's separating value into a single 3-node

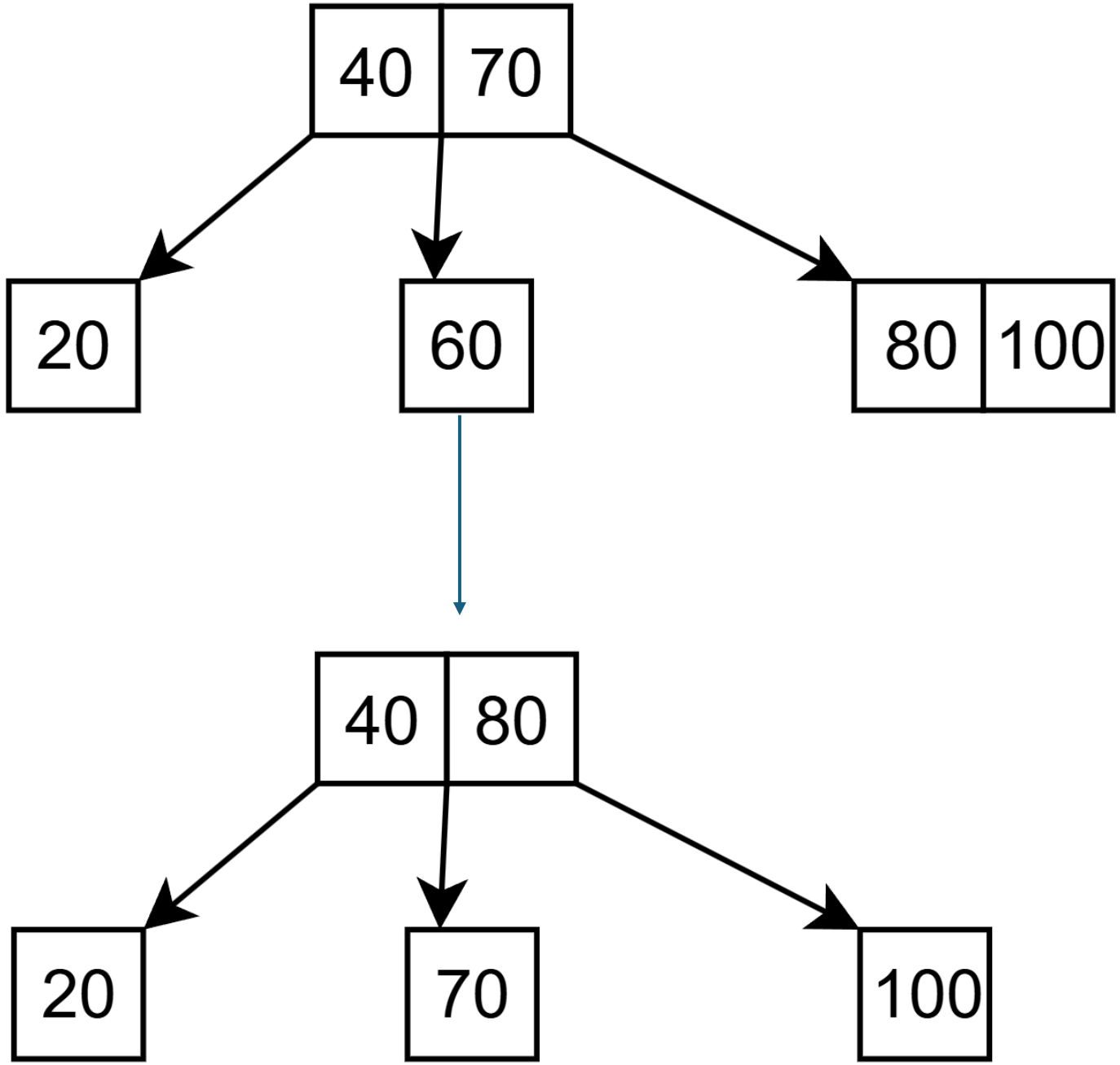
2-3 Trees

- Deletion: Delete 50.
- Just replace 50 with its inorder predecessor



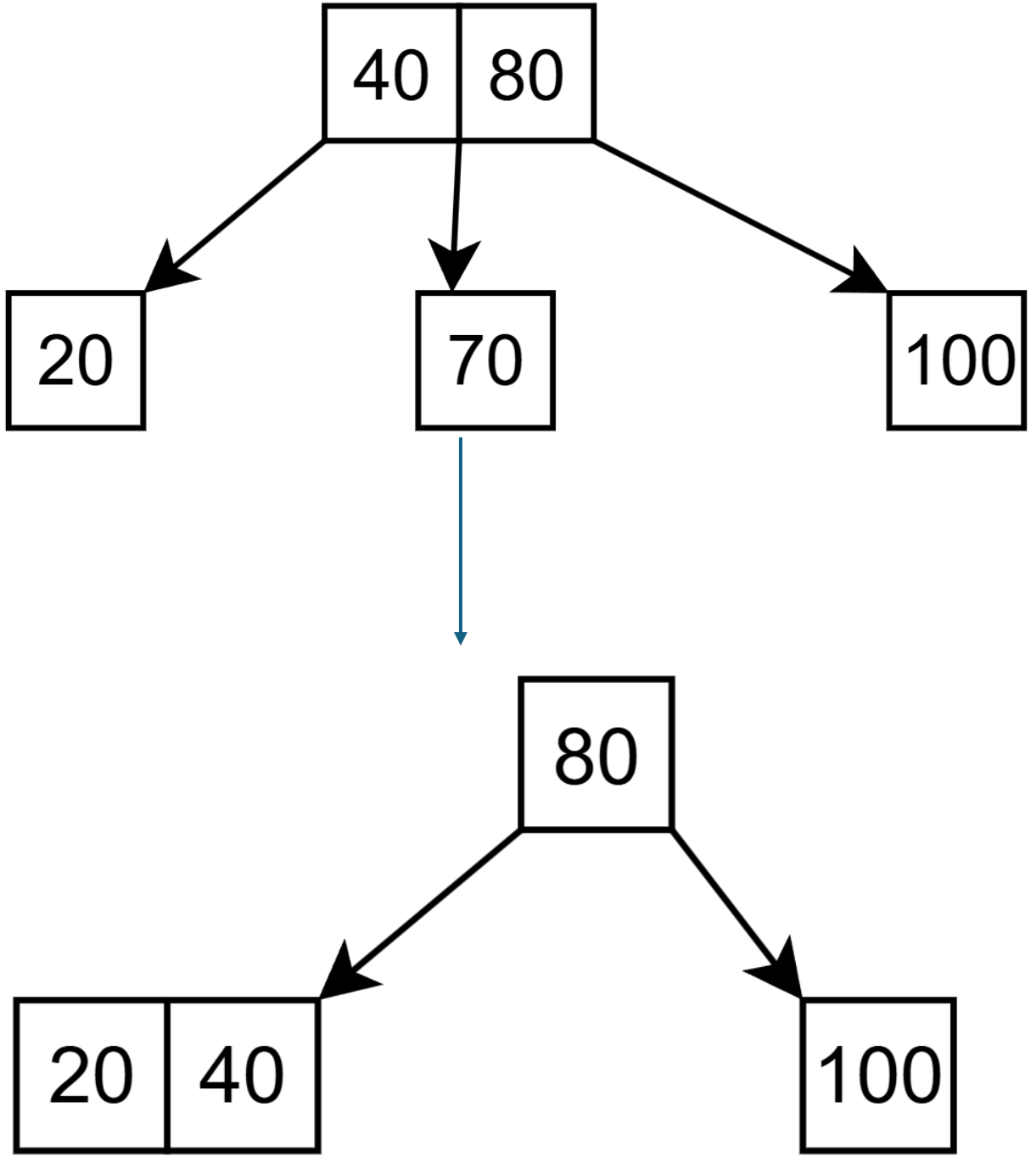
2-3 Trees

- Deletion: Delete 60.
- Else if the leaf has an adjacent sibling 3-node:
- Borrow a key from it (via the parent's separating value)



2-3 Trees

- Deletion: Delete 70.
- Else: (leaf was 2-node, its adjacent siblings are also 2-nodes)
- Collapse the node, one sibling, and the parent's separating value into a single 3-node



2-3-4 Trees

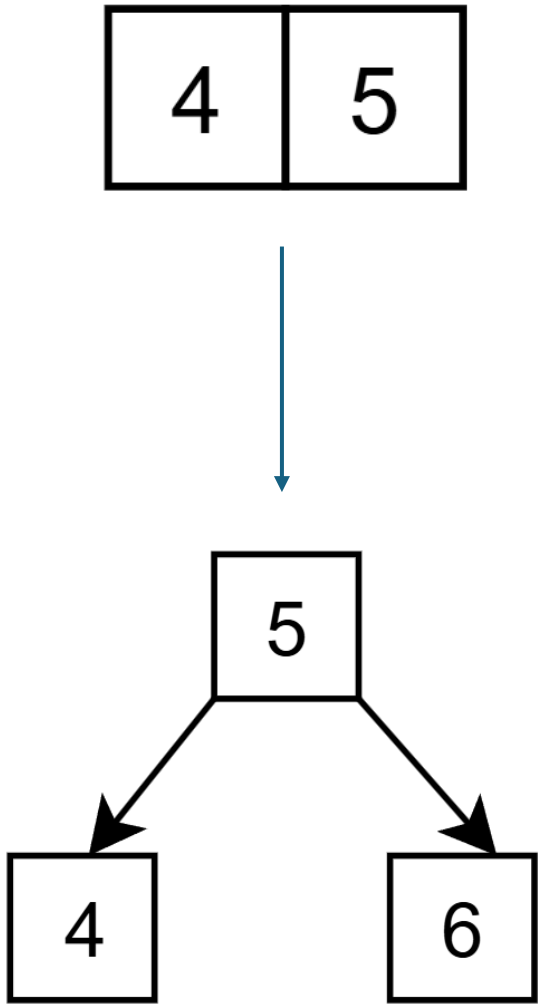
- If two values per node is good, is three better?
- Maybe: you get a shallower tree (more data per level)… But more comparisons per node
- A Red-Black tree is a binary tree implementation of a 2-3-4 tree

2-3-4-5-… Trees

- How far can we take this idea of adding data to each node?
- When we store the data on disk, the node size can be made to fit the block size of the disk
- This idea leads to a *B-Tree*, which is used for indexing in databases, and more generally in file systems

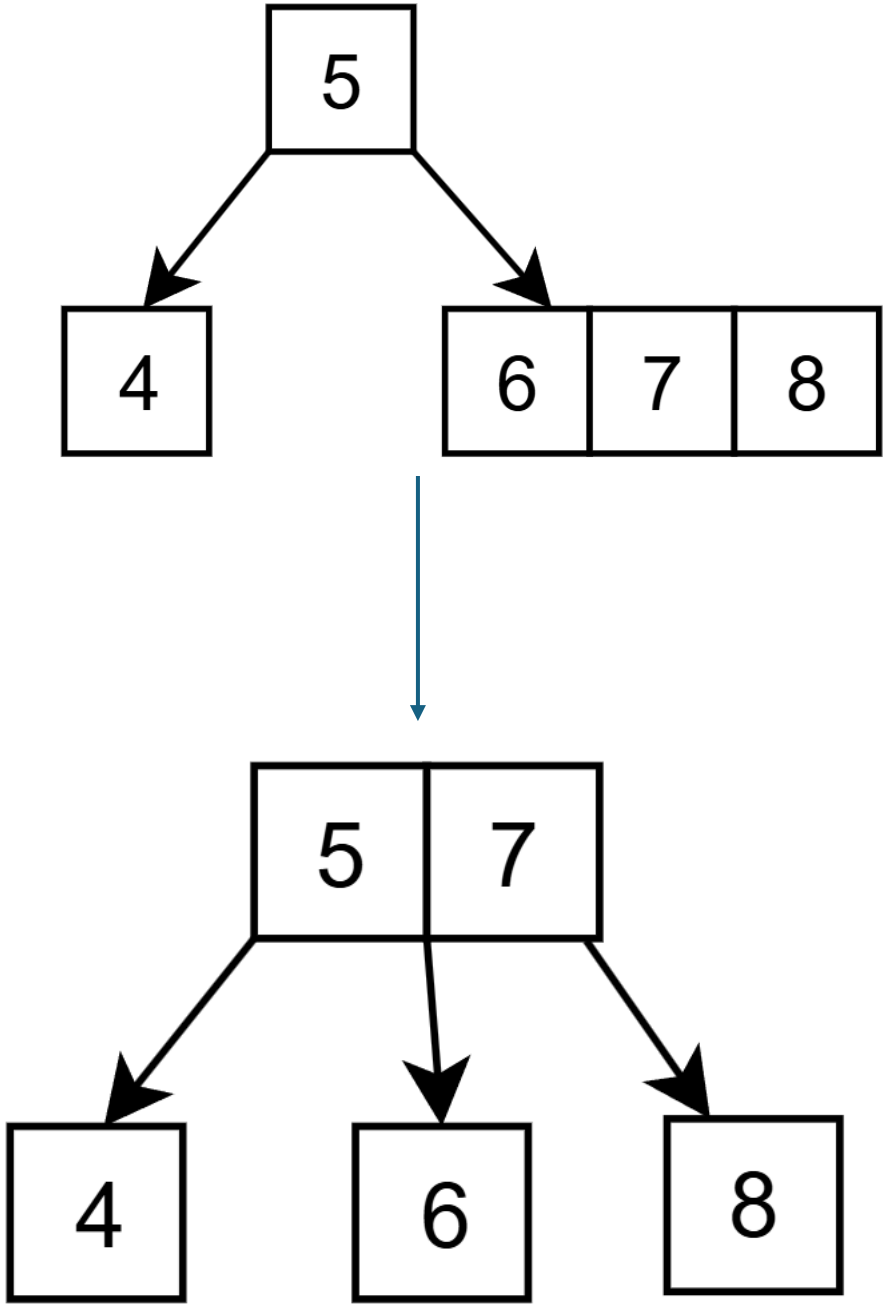
2-3 Trees

- Before we move on, let's get some practice in. Build a 2-3 Tree using the following values (in order): 4, 5, 6, 7, 8, 9, and 10



2-3 Trees

- Before we move on, let's get some practice in. Build a 2-3 Tree using the following values (in order): 4, 5, 6, 7, 8, 9, and 10



2-3 Trees

- Before we move on, let's get some practice in. Build a 2-3 Tree using the following values (in order): 4, 5, 6, 7, 8, 9, and 10

